
Exercises Advanced Lesson 11

Your Name: Mikey Ferguson

Your Math Class: MATH 7013 - Advanced Mathematical Modeling

Pattern matching in functional arguments:

When you are instructed to construct a function whose argument will match exactly one arbitrary object, you will write: $f[x_]$

If you want delayed assignment to that function you will write: $f[x_] := rhs[x]$

1: Write a delayed assignment function of exactly one argument which will square the argument.

Evaluate: $?f$

Evaluate: $f[2]$

Evaluate: $f[x]$

Evaluate: $f[y] \Rightarrow y^2$

Evaluate: $f["string"]$

Evaluate: $f[\{1,2,3\}]$

Evaluate: $f\left[\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}\right] \Rightarrow \{\{1,4\},\{9,16\}\}$

Evaluate: $f[]$

Evaluate: $f[x,y]$

Evaluate: $f[1,2,3,4]$

Explain what is different about the last three evaluations.

```
ClearAll[f, x, y];
```

```
f[x_] := x2
```

```
?f
```

```
{f[2], f[x], f[y], f["string"], f[{1, 2, 3}]}
```

```
f[{{1, 2}, {3, 4}}]
```

```
{f[], f[x, y], f[1, 2, 3, 4]}
```

$f[x_]$ takes exactly one argument. $f[]$, $f[x, y]$ and $f[1,2,3,4]$ supply the wrong number, so no definition matches and they come back unevaluated.

2: Write a delayed assignment function of EXACTLY ZERO arguments which will take on the value 3.

Evaluate: $?f$

Evaluate: $f[] \Rightarrow 3$

Evaluate: $f[2]$

Evaluate: $f[x]$

Evaluate: $f[x, y]$

Evaluate: $f[1, 2, 3, 4]$

Evaluate: $f\left[\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}\right]$

Explain what is different about the last five evaluations.

```
ClearAll[f, x, y];
```

```
f[] := 3
```

```
?f
```

```
f[]
```

```
{f[2], f[x], f[x, y], f[1, 2, 3, 4], f[{1, 2}, {3, 4}]}
```

$f[]$ is defined for exactly zero arguments, so it gives 3. The other five all supply at least one argument, match nothing, and stay unevaluated.

3: Write a delayed assignment function of ONE OR MORE arguments which will square the LIST of arguments.

Does the square (i.e. Power) have attribute Listable? What does this mean?

Evaluate: $?f$

Evaluate: $f[2]$

Evaluate: $f[x]$

Evaluate: $f[y] \Rightarrow \{y^2\}$

Evaluate: $f["string"]$

Evaluate: $f[1, 2, 3] \Rightarrow \{1, 4, 9\}$

Evaluate: $f\left[\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}\right]$

Evaluate: $f[x, y]$

Evaluate: $f[1, 2, 3, 4]$

Evaluate: $f[x, 1, 2, 3] \Rightarrow \{x^2, \{1, 4, 9\}\}$

Evaluate: $f[x, 1, 2, 3, \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}]$

Evaluate: $f[]$

Explain what is different about the last evaluation.

```
ClearAll[f, x, y];
```

```
f[x_] := {x}^2
```

```
Attributes[Power]
```

```
?f
```

```
{f[2], f[x], f[y], f["string"], f[{1, 2, 3}]}
```

```
{f[{1, 2}, {3, 4}], f[x, y], f[1, 2, 3, 4]}
```

`f[x, {1, 2, 3}]`

`f[x, {1, 2, 3}, {{1, 2}, {3, 4}}]`

`f[]`

`x__` means one or more arguments, and `{x}^2` squares the list of them. Power has the attribute `Listable`, which means it applies elementwise, so `{1,2,3}^2` is `{1,4,9}`. The last call, `f[]`, fails because `__` requires at least one argument.

4: Write a delayed assignment function of ZERO OR MORE arguments which will square the LIST of arguments.

Evaluate: `? f`

Evaluate: `f[2]`

Evaluate: `f[x]`

Evaluate: `f[y]`

Evaluate: `f["string"]`

Evaluate: `f[{1,2,3}]` $\Rightarrow \{\{1,4,9\}\}$

Evaluate: `f[ $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ ]`

Evaluate: `f[x,y]`

Evaluate: `f[1,2,3,4]`

Evaluate: `f[x, {1,2,3}]` $\Rightarrow \{x^2, \{1, 4, 9\}\}$

Evaluate: `f[x, {1,2,3},  $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ ]`

Evaluate: `f[]`

Explain the last evaluation.

`ClearAll[f, x, y];`

`f[x__] := {x}^2`

`? f`

`{f[2], f[x], f[y], f["string"], f[{1, 2, 3}]}`

`{f[{{1, 2}, {3, 4}}], f[x, y], f[1, 2, 3, 4]}`

`f[x, {1, 2, 3}]`

`f[x, {1, 2, 3}, {{1, 2}, {3, 4}}]`

`f[]`

`x__` means zero or more arguments, so `f[]` now matches. The argument list is empty, and the empty list squared is still the empty list, so the answer is `{}`.

5: Write a delayed assignment function of exactly TWO arguments which will square the first argument times the Sine of the second argument.

Evaluate: `? f`

Evaluate: `f[]`

Evaluate: `f[x]`

Evaluate: $f[\{1,2\}]$
 Evaluate: $f[\{1,2,3\}]$
 Evaluate: $f[1,2,3,4]$
 Evaluate: $f[x,y] \Rightarrow x^2 \sin[y]$
 Evaluate: $f[x, \{1,2,3\}]$
 Evaluate: $f["string", \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}]$

Explain why the first five evaluations did not work.

```
ClearAll[f, x, y];
f[x_, y_] := x^2 Sin[y]
?f
{f[], f[x], f[{1, 2}], f[{1, 2, 3}], f[1, 2, 3, 4]}
f[x, y]
f[x, {1, 2, 3}]
f["string", {{1, 2}, {3, 4}}]
```

$f[x_, y_]$ needs exactly two arguments. The first five calls supply zero, one or four, or supply a single list where two separate things were wanted, so none of them match.

6: Part 1: Write a delayed assignment function of exactly THREE arguments which equals the list of those three arguments.

Part 2: Write a delayed assignment function of exactly FOUR arguments which equals the list of those four arguments.

```
ClearAll[f3, f4, x, y, z, w];
f3[x_, y_, z_] := {x, y, z};
f4[x_, y_, z_, w_] := {x, y, z, w}
{f3[1, 2, 3], f4[1, 2, 3, 4]}
{f3[1, 2], f4[1, 2, 3]}
```

$f3$ takes exactly three arguments and $f4$ exactly four. Given the right number they return the list of their arguments; given the wrong number nothing matches and they stay unevaluated.

7: Write a delayed assignment function of exactly ONE argument followed by ZERO or more arguments which will square the first argument times the Sine of the LIST of the second arguments.

```
Evaluate: ?f
Evaluate: f[]
Evaluate: f[x]  $\Rightarrow \{ \}$ , How did Mathematica calculate this value?
Evaluate: f[{1,2}] Why did Mathematica fail to calculate this value?
Evaluate: f[x,y]  $\Rightarrow \{x^2 \sin[y]\}$ 
Evaluate: f[2,3,4,5]  $\Rightarrow \{4 \sin[3], 4 \sin[4], 4 \sin[5]\}$ 
Evaluate: f[x, y, z, w]
```

Evaluate: `f[x, {1,2,3}, A, B]` Look at this result in: ColumnForm,
How did *Mathematica* calculate this value?

Evaluate: `f["string",  $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}]$`

```
ClearAll[f, x, y, z, w, A, B];
```

```
f[x_, y___] := x^2 Sin[{y}]
```

```
?f
```

```
f[]
```

```
f[x]
```

```
f[{1, 2}]
```

```
f[x, y]
```

```
f[2, 3, 4, 5]
```

```
f[x, y, z, w]
```

```
f[x, {1, 2, 3}, A, B]
```

```
ColumnForm[f[x, {1, 2, 3}, A, B]]
```

```
f["string", {{1, 2}, {3, 4}}]
```

`f[x]` supplies no later arguments, so the second group is empty, Sin of the empty list is the empty list, and the answer is `{}`. `f[{1,2}]` has an empty second group for the same reason, but this time the first argument is a two-element list, and Mathematica cannot multiply a two-element list by an empty one, so it reports a length mismatch and stops. The pattern matched perfectly well; it is the arithmetic that failed. `f[x, {1,2,3}, A, B]` shows the grouping clearly: everything after the first argument is collected into one list and Sin spreads over it.

8: Write a delayed assignment function of the Sine of exactly one argument which will square the argument which appears inside the Sine function.

Evaluate: `?f`

Evaluate: `f[Sin[2]]` $\Rightarrow 4$

Evaluate: `f[Sin[y]]`

Evaluate: `f[Sin[{1,2,3}]]` Why did *Mathematica* fail to match this pattern?

Evaluate: `f[Sin[Log[x]]]`

Evaluate: `f[x]`

Evaluate: `f[Cos[y]]`

Evaluate: `f[1,2,3,4]`

Explain what is different about the last three evaluations.

```
ClearAll[f, x, y];
```

```
f[Sin[x_]] := x^2
```

```
?f
```

```
{f[Sin[2]], f[Sin[y]], f[Sin[Log[x]]]}
```

```
f[Sin[{1, 2, 3}]]
```

```
{f[x], f[Cos[y]], f[1, 2, 3, 4]}
```

The pattern is $f[\text{Sin}[x_]]$, so the argument must literally be a sine of one thing. $f[\text{Sin}[\{1,2,3\}]]$ fails because Sin is Listable and spreads over the list before f ever sees it, so the argument that arrives is $\{\text{Sin}[1], \text{Sin}[2], \text{Sin}[3]\}$, a list and not a sine. $f[x]$ and $f[\text{Cos}[y]]$ are not sines either, and $f[1,2,3,4]$ has four arguments.

9: Write a delayed assignment function which will have several different assignments. Specifically, $f[\text{function}[x_]] := \text{derivative}[x]$ for different functions ("f" will be defined using multiple assignments. Also, "f" will NOT use the chain rule correctly, i.e. $f[\text{Sin}[2x]] \Rightarrow \text{Cos}[2x]$ NOT $2 \text{Cos}[2x]$). "f" should use pattern matching to accept as arguments the functions: $\text{Sin}[x]$, $\text{Cos}[x]$, $\text{Log}[x]$, E^x and return the corresponding "derivative" functions.

```
Evaluate: ? f
```

```
Evaluate: f[ $e^x$ ]
```

```
Evaluate: f[Sin[y]]  $\Rightarrow \text{Cos}[y]$ 
```

```
Evaluate: f[Cos[x + y]]
```

```
Evaluate: f[Sin[2]]  $\Rightarrow \text{Cos}[2]$ 
```

```
Evaluate: f[Sin[Log[x]]]
```

```
Evaluate: f[Sin[{1,2,3}]] Why did Mathematica fail to match this pattern?
```

```
ClearAll[f, x, y];
```

```
f[Sin[x_]] := Cos[x];
```

```
f[Cos[x_]] := -Sin[x];
```

```
f[Log[x_]] :=  $\frac{1}{x}$ ;
```

```
f[ $e^x$ ] :=  $e^x$ 
```

```
?f
```

```
{f[ $e^x$ ], f[Sin[y]], f[Cos[x + y]], f[Sin[2]]}
```

```
f[Sin[Log[x]]]
```

```
f[Sin[{1, 2, 3}]]
```

Each rule matches one named function and returns its derivative. $f[\text{Sin}[\text{Log}[x]]]$ gives $\text{Cos}[\text{Log}[x]]$: the rule matches the outer sine and ignores what is inside, which is exactly why there is no chain rule here. $f[\text{Sin}[\{1,2,3\}]]$ fails for the earlier reason, that Sin spreads over the list first and leaves no sine to match.

10: Odd functions have the property that: $f(-x) \Rightarrow -f(x)$. Write a delayed assignment function which uses pattern matching which will make this assignment. (Note that this is a recursive assignment.)

```
Evaluate: ? f
```

Evaluate: `f[-x]`

Evaluate: `f[-y]` $\Rightarrow$ `-f[y]`

Evaluate: `f[-Sin[y]]`

Evaluate: `f[-5]` Why did *Mathematica* fail to match this pattern?

Hint: look at `FullForm[-5]` and `FullForm[-x]`

```
ClearAll[f, x, y];
```

```
f[-x_] := -f[x]
```

```
?f
```

```
{f[-x], f[-y], f[-Sin[y]]}
```

```
f[-5]
```

```
{FullForm[-5], FullForm[-x]}
```

`f[-x]` works because `-x` is stored as `Times[-1, x]`, which fits the pattern `-x_`. `f[-5]` fails because `-5` is stored as the single number `-5`, not as `Times[-1, 5]`, so there is no product there for the pattern to match.

`FullForm` shows the difference plainly.

11: How would you add a new assignment to the function "f" in Problem 10 so that the function "f" with two assignments would also accept the pattern: $f(-5) \Rightarrow -f(5)$ for different negative numbers? (There are several correct ways to do this.)

Evaluate: `?f`

Evaluate: `f[-x]` $\Rightarrow$ `-f[x]`

Evaluate: `f[x]`

Evaluate: `f[-y]`

Evaluate: `f[-Sin[y]]`

Evaluate: `f[-5]` $\Rightarrow$ `-f[5]`

Evaluate: `f[-7]`

Evaluate: `f[5]`

```
ClearAll[f, x, y];
```

```
f[-x_] := -f[x];
```

```
f[n_?Negative] := -f[-n]
```

```
?f
```

```
{f[-x], f[x], f[-y], f[-Sin[y]]}
```

```
{f[-5], f[-7], f[5]}
```

12: Write a delayed assignment function using pattern matching that will accept as its argument a list (of 3 or more elements) which will return the value of the third element in the list.

Evaluate: `?f`

Evaluate: `f[]` Why did *Mathematica* fail to match this pattern?

Evaluate: `f[x]` Why did *Mathematica* fail to match this pattern?

Evaluate: `f[{1,2}]` Why did *Mathematica* fail to match this pattern?

Evaluate: $f[\{1,2,3\}] \Rightarrow 3$

Evaluate: $f[\{1,2,3,4\}]$

Evaluate: $f[\{1,2,3,4,5\}]$

Evaluate: $f[\{x,y,\{1,2,3\},A,B\}]$

```
ClearAll[f, x, y, A, B];
```

```
f[{_, _, c_, ____}] := c
```

```
?f
```

```
{f[], f[x], f[{1, 2}]}
```

```
{f[{1, 2, 3}], f[{1, 2, 3, 4}], f[{1, 2, 3, 4, 5}]}
```

```
f[{x, y, {1, 2, 3}, A, B}]
```

The pattern asks for a list of at least three elements. $f[]$ has no argument, $f[x]$ has one that is not a list, and $f[\{1,2\}]$ is a list that is too short, so none of the three match.

13: Write a delayed assignment function using pattern matching that will accept as its argument:

A list of elements,

Which contains another list (of 1 or more elements)

Which will return the inside list of elements.

Evaluate: $?f$

Evaluate: $f[1, 2, \{4, 5\}, 6]$

Why did *Mathematica* fail to match this pattern?

Evaluate: $f[\{1, 2, 3, 4\}]$

Why did *Mathematica* fail to match this pattern?

Evaluate: $f[\{1, 2, \{\}, 3, 4\}]$

Why did *Mathematica* fail to match this pattern?

Evaluate: $f[\{\{1\}, 2, 3, 4\}] \Rightarrow \{1\}$

Evaluate: $f[\{1, 2, \{3\}, 4\}] \Rightarrow \{3\}$

Evaluate: $f[\{1, \{2, 3\}, 4\}] \Rightarrow \{2, 3\}$

Evaluate: $f[\{1, 2, 3, \{4\}\}] \Rightarrow \{4\}$

Evaluate: $f[\{1, 2, \{3, 4, 5\}\}]$

Evaluate: $f[\{\{1, 2, 3, 4, 5\}\}]$

Evaluate: $f[\{1, 2, \{3, \{4, 5\}\}\}]$

Evaluate: $f[\{x, y, \{1,2,3\}, A, B\}]$

```
ClearAll[f, x, y, A, B];
```

```
f[{____, l_List, ____}] := l
```

```
?f
```

```
f[1, 2, {4, 5}, 6]
```

```
f[{1, 2, 3, 4}]
```

```
f[{1, 2, {}, 3, 4}]
```

```
{f[\{\{1\}, 2, 3, 4\}], f[\{1, 2, \{3\}, 4\}], f[\{1, \{2, 3\}, 4\}], f[\{1, 2, 3, \{4\}\}]}]
```

```
{f[\{1, 2, \{3, 4, 5\}\}], f[\{\{1, 2, 3, 4, 5\}\}]}]
```

```
f[{1, 2, {3, {4, 5}}}]
```

```
f[{x, y, {1, 2, 3}, A, B}]
```

The pattern asks for a list that contains a list. `f[1, 2, {4,5}, 6]` fails because it passes four separate arguments rather than one list. `f[{1,2,3,4}]` fails because nothing inside it is a list. `f[{1, 2, {}, 3, 4}]` does match in this version, because the empty list is still a list, so the answer is the empty list.

14: Calculate the `FullForm[...]` of each expression inside the quotation marks:

```
"_" ⇒ Blank[]
```

```
"x_"
```

```
"x__"
```

```
"x___"
```

```
"x_Integer"
```

```
"x_?EvenQ"
```

```
"x_?(# > 3 &)"
```

```
"x_Integer?EvenQ"
```

Create a function which will return the character string "Positive Integer" when its argument is a positive integer and have the function remain unevaluated under all other circumstances. Your function should use pattern matching similar to the last entry given above:

```
f[x_...?...] := "Positive Integer"
```

Evaluate:

```
f[3]
```

```
f[3.45]
```

```
f[-3]
```

Pattern matching in replacement rules:

When you are instructed to construct an immediate replacement rule you will write: `lhs → rhs`

When you are instructed to construct a delayed replacement rule you will write: `lhs ⇨ rhs`

When you are instructed to apply the replacement rule to an expression you will write:

```
expression( ... lhs ... ) /. lhs → rhs
```

which will yield:

```
expression( ... rhs ... )
```

```
Clear[x];
```

```
Column[List @@ Function[u, ToString[FullForm[u]]] /@
```

```
Hold[, x_, x__, x___, x_Integer, x_?EvenQ, x_?(#1 > 3 &), x_Integer?EvenQ]]
```

```
ClearAll[f];
```

```
f[x_Integer?Positive] := "Positive Integer"
```

```
{f[3], f[3.45`], f[-3]}
```

14: Write an immediate replacement rule which pattern matches 2 and replaces it with 222 and assign this rule to: `rule1`. Apply this replacement rule to the following expressions:

Evaluate: ?rule1 $\Rightarrow$ rule1 = $2 \rightarrow 222$
 Evaluate: {1, 2, 3, 4} /. rule1 $\Rightarrow$ {1, 222, 3, 4}
 Evaluate: Sin[2] /. rule1
 Evaluate: {1, 2, Sin[2], x^2 } /. rule1
 Evaluate: {1, 2, 3, 2+2} /. rule1 Why was the last replacement missed?
 Evaluate: {1, 2, 3, Hold[2+2]} /. rule1
 Evaluate: 2 Sin[2 Sin[2 Sin[2 Sin[2 x]]]] /. rule1

ClearAll[rule1, x];

rule1 = $2 \rightarrow 222$

?rule1

{1, 2, 3, 4} /. rule1

Sin[2] /. rule1

{1, 2, Sin[2], x^2 } /. rule1

{1, 2, 3, 2+2} /. rule1

{1, 2, 3, Hold[2+2]} /. rule1

2 Sin[2 Sin[2 Sin[2 Sin[2 x]]]] /. rule1

In {1, 2, 3, 2+2} the 2+2 was added up to 4 before the rule was ever applied, so by then there was no 2 left in that position to replace. Wrapping it as Hold[2+2] stops the addition, and then both 2s are replaced.

15: Write an immediate replacement rule which pattern matches $(x_)^3$ and replaces it with x^{333} and assign this rule to: rule2. Apply this replacement rule to the following expressions:

Evaluate: ?rule2 $\Rightarrow$ rule2 = $(x_)^3 \rightarrow x^{333}$

Evaluate: $\{x^2, x^3, x^4, y^3, \text{Sin}[x]^3, \frac{1}{2+z^3}, (x+y)^3\}$ /. rule2

Evaluate: $\left(\frac{1}{2+y}\right)^3$ /. rule2 Why was the last replacement missed?

Look at the following to answer the above question:

Evaluate: FullForm[x^3]

Evaluate: FullForm[(2+y)³]

Evaluate: Hold[$\left(\frac{1}{2+y}\right)^3$] /. rule2

Evaluate: FullForm[$\left(\frac{1}{2+y}\right)^3$]

ClearAll[rule2, x, y, z];

rule2 = $x_^3 \rightarrow x^{333}$

?rule2

$\{x^2, x^3, x^4, y^3, \text{Sin}[x]^3, \frac{1}{2} + z^3, (x+y)^3\}$ /. rule2

$$\left(\frac{1}{2} + y\right)^3 /. \text{rule2}$$

$$\{\text{FullForm}[x^3], \text{FullForm}[(2 + y)^3], \text{FullForm}\left[\left(\frac{1}{2} + y\right)^3\right]\}$$

$$\text{Hold}\left[\left(\frac{1}{2} + y\right)^3\right] /. \text{rule2}$$

The rule matches any cube. In this version $(1/2 + y)^3$ is matched as well and becomes $(1/2 + y)^{333}$. The exercise expects it to be missed, but the full form is `Power[Plus[Rational[1,2], y], 3]`, with a plain 3 in the exponent, so it fits the pattern.

16: Make a list of the previous two rules. Apply this list of replacement rules to the following expressions:

Evaluate: `? {rule1, rule2}`

Evaluate: $\{x^2, x^3, x^4, y^3, \sin[x]^3, \frac{1}{2+z^3}, (x+y)^3\} /. \{\text{rule1}, \text{rule2}\}$

Evaluate: $(2 + y)^3 /. \{\text{rule1}, \text{rule2}\}$ Why was the first replacement missed?

Evaluate: $(2 + y)^3 //. \{\text{rule1}, \text{rule2}\}$ Why did this work?

`{rule1, rule2}`

$$\{x^2, x^3, x^4, y^3, \sin[x]^3, \frac{1}{2} + z^3, (x + y)^3\} /. \{\text{rule1}, \text{rule2}\}$$

$$(2 + y)^3 /. \{\text{rule1}, \text{rule2}\}$$

$$(2 + y)^3 //. \{\text{rule1}, \text{rule2}\}$$

With `/. Mathematica` works from the outside in and does not re-scan what it has just replaced. The cube rule matches the whole expression first, so the 2 buried inside is never visited and rule1 is missed. `//.` keeps re-applying the rules until nothing changes, so the 2 is caught on a later pass and the answer becomes $(222 + y)^{333}$.

17: The following example illustrates some of the power of pattern matching and replacement rules. As you already know, every operation in *Mathematica* has an internal functional representation.

For example, generate the plot of x^2 and assign the output to: `plot1`.

Evaluate: `plot1 = Plot[x^2, {x, 0, 1}]`

To display the internal functional representation:

Evaluate: `? plot1`

Observe that the contents of `plot1` is something like:

`plot1 = Graphics[{{{ }, { }, {Hue[0.67, 0.6, 0.6], Line[{{2.04082 × 10-8, 4.16493 × 10-16}, .....}`

along with a list of replacement rules for graphic options. The first part consists of the Line command, which just connects a sequence of line segments through a list of coordinates $\{\{x_1, y_1\}, \{x_2, y_2\}, \dots\}$.

Our objective is to replace the line segments with a list of points using pattern matching and replacement rules. To accomplish this, we need to understand the pattern in the Line command as well as the Point command. The patterns are as follows:

The pattern we have: `Line [{x1,y1}, {x2,y2}, ... ]`

The pattern we want: `{Point [{x1,y1}], Point [{x2,y2}], ... }`

Thus, we want to use pattern matching and replacement rules to replace:

`Line [{ ... }, {x,y}, ... ]`

with:

`{ ... }, Point [{x,y}], ... }`

This can be accomplished in two steps. Replace the content in plot1 by creating a replacement rule from:

`Line [{ ... }, {x,y}, ... ] → Line [{ ... }, Point [{x,y}], ... ]`

to get an intermediate step (call it plot2). plot2 will NOT be a proper plot, so it can not be displayed. The graphic display can be suppressed by ending with a ";". Then to see the content of plot2:

Evaluate: `? plot2`

Then replace the content in plot2 by creating a replacement rule from:

`Line [{ ... }, Point [{x,y}], ... ] → { ... }, Point [{x,y}], ... }`

to get our final graphic commands (call it plot3). Then to see the content of plot3:

Evaluate: `? plot3`

plot3 will be a proper plot, so it can be displayed.

Evaluate: plot3

You should conclude that EVERY *Mathematica* expression, no matter what its form, can be manipulated in this way.

```
ClearAll[x, plot1, plot2, plot3];
plot1 = Plot[x2, {x, 0, 1}]
Length[Cases[plot1, _Line, ∞]]

plot2 = plot1 /. Line[{pts__}] :> Line[Point /@ {pts}];
Length[Cases[plot2, _Point, ∞]]

plot3 = plot2 /. Line[{pts__ Point}] :> {pts};
{Head[plot3], Length[Cases[plot3, _Line, ∞]], Length[Cases[plot3, _Point, ∞]]}

plot3
```